

Code Explanation

Data Processing Pipeline Explanation

The given code is a data processing pipeline written in Python using the PySpark library. It reads customer and order data from CSV files, cleans and processes the data, and then writes it to Delta tables. The pipeline also creates or updates an ordersummary table using Slowly Changing Dimension (SCD) Type 2 and generates a customer aggregate spend table.

Overview of the Code

The code is structured into several functions, each responsible for a specific task in the data processing pipeline. The main function orchestrates the entire process, from creating a Spark session to writing the processed data to Delta tables.

Step 1: Creating a Spark Session

This step creates a Spark session, which is the entry point to any functionality in Spark. The `create\_spark\_session` function returns a Spark session with the app name "Customer Order Processing" and Hive support enabled.

```
return SparkSession.builder
    .appName("Customer Order Processing")
    .enableHiveSupport()
    .getOrCreate()
```

Step 2: Reading Source Data

The `read\_source\_data` function reads customer and order data from CSV files stored in volumes. It uses the Spark session to read the data and returns two DataFrames: `customer\_df` and `order\_df`.

```
customer_df = spark.read.format("csv")
    .option("header", "true")
    .option("inferSchema", "true")
    .load("/Volumes/gen_ai_poc_databrickscoe/sdlc_wizard/customerdata")

order_df = spark.read.format("csv")
    .option("header", "true")
    .option("inferSchema", "true")
    .load("/Volumes/gen_ai_poc_databrickscoe/sdlc_wizard/orderdata")
```

Step 3: Cleaning Data

The `clean\_data` function cleans the customer and order data by removing duplicates and null values. It also adds a `TotalAmount` column to the order data.

```
clean_customer_df = customer_df
    .dropDuplicates(["CustId"])
    .filter(
        (F.col("CustId").isNotNull()) &
        (F.col("Name").isNotNull()) &
        (F.col("EmailId").isNotNull()) &
        (F.col("Region").isNotNull())
```

```

    )
clean_order_df = order_df
    .dropDuplicates(["OrderId"])
    .filter(
        (F.col("OrderId").isNotNull()) &
        (F.col("ItemName").isNotNull()) &
        (F.col("PricePerUnit").isNotNull()) &
        (F.col("Qty").isNotNull()) &
        (F.col("Date").isNotNull()) &
        (F.col("CustId").isNotNull())
    )
    .withColumn("TotalAmount", F.col("PricePerUnit") * F.col("Qty"))

```

Step 4: Creating or Updating the Ordersummary Table

The `create\_or\_update\_ordersummary` function creates or updates the ordersummary table using SCD Type 2. It checks if the table exists, and if it does, it expires old records for changed customers and inserts new records. If the table does not exist, it creates it with all records as active.

```

if table_exists:
    # Expire old records and insert new records for changed customer
    s
    delta_table = DeltaTable.forName(spark, "gen_ai_poc_databricksco
e.sdmc_wizard.ordersummary")
    delta_table.alias("t").merge(
        changed_customers.alias("s"),
        "t.CustId = s.CustId AND t.IsActive = true"
    ).whenMatchedUpdate(
        set={
            "IsActive": "false",
            "EndDate": "current_timestamp()"
        }
    ).execute()

    new_records = clean_customer_df.alias("c")
        .join(
            changed_customers.alias("cc"),
            "CustId",
            "inner"
        )
        .join(
            clean_order_df.alias("o"),
            "CustId",
            "inner"
        )
        .select(
            F.col("c.CustId"),
            F.col("c.Name"),
            F.col("c.EmailId"),
            F.col("c.Region"),
            F.col("o.OrderId"),
            F.col("o.ItemName"),
            F.col("o.PricePerUnit"),
            F.col("o.Qty"),
            F.col("o.Date"),
            F.col("o.TotalAmount"),

```

```

        F.lit(True).alias("IsActive"),
        current_timestamp.alias("StartDate"),
        F.lit(None).cast("timestamp").alias("EndDate")
    )
else:
    # Create the ordersummary table with all records as active
    joined_df = clean_customer_df.alias("c")
        .join(
            clean_order_df.alias("o"),
            "CustId",
            "inner"
        )
        .select(
            F.col("c.CustId"),
            F.col("c.Name"),
            F.col("c.EmailId"),
            F.col("c.Region"),
            F.col("o.OrderId"),
            F.col("o.ItemName"),
            F.col("o.PricePerUnit"),
            F.col("o.Qty"),
            F.col("o.Date"),
            F.col("o.TotalAmount"),
            F.lit(True).alias("IsActive"),
            current_timestamp.alias("StartDate"),
            F.lit(None).cast("timestamp").alias("EndDate")
        )
    )

```

Step 5: Creating the Customer Aggregate Spend Table

The `create\_customer\_aggregate\_spend` function creates the customeraggregatespend table by aggregating data from the ordersummary table.

```

agg_df = spark.table("gen_ai_poc_databrickscoe.sdmc_wizard.ordersumm
ary")
    .filter(F.col("IsActive") == True)
    .groupBy("Name", "Date")
    .agg(F.sum("TotalAmount").alias("TotalAmount"))

agg_df.write
    .format("delta")
    .mode("overwrite")
    .option("overwriteSchema", "true")
    .saveAsTable("gen_ai_poc_databrickscoe.sdmc_wizard.customeraggre
gatespend")

```

Step 6: Orchestrating the Data Processing Pipeline

The `main` function orchestrates the entire data processing pipeline by calling the various functions in sequence.

```

spark = create_spark_session()
customer_df, order_df = read_source_data(spark)
clean_customer_df, clean_order_df = clean_data(customer_df, order_df
)

```

```
create_or_update_ordersummary(spark, clean_customer_df, clean_order_df)
create_customer_aggregate_spend(spark)
```